Government Engineering College Kishanganj

Department of Computer Science and Engineering
Computer Networks LAB-02
Exploring Packet Sniffer and Packet Analyzer

Name	Achintya Singh
Registration No.	24105142034
Subject	Computer Networks
College	Government Engineering College Kishanganj

Procedure followed

Wireshark was installed and set to capture on the active network interface. All browser tabs were closed before starting the capture. The capture was started, then **http://neverssl.com/** was opened, the page was left idle for 2 minutes, and then on the same tab **https://www.cornell.edu/** was visited. The browser tabs were closed, the capture was stopped, and the trace was saved as a .pcap file (lab03.pcap) and analysed using Wireshark / tshark.

Question 1 [3 marks] — Protocols observed at each layer

The protocol hierarchy was inspected using **Statistics** → **Protocol Hierarchy** in Wireshark. The following protocols were observed:

Layer	Protocols Observed
A. Application layer	DNS, HTTP, TLS/SSL, mDNS, SSDP (and QUIC, which carries application data over UDP)
B. Transport layer	TCP, UDP
C. Network layer	IPv4, IPv6, ARP, ICMPv6

Note: ARP and ICMPv6 are technically network-layer support protocols (address resolution and control-message signalling respectively) and were seen alongside plain IPv4/IPv6 traffic.

Question 2 [2 marks] — Total data received

A. When accessing http://neverssl.com/

The neverssl.com session runs entirely over a single TCP connection (TCP stream, port 80). Summing the frame lengths of every packet in that stream gives:

```
Total data received ≈ 5,618 bytes (on-wire), of which 3,942 bytes is TCP payload (the rest is Ethernet/IP/TCP header overhead).
```

B. When accessing https://www.cornell.edu

The Cornell homepage is loaded over TLS/QUIC and pulls in several sub-resources (fonts from Typekit, Google Analytics/Tag Manager, etc.) in addition to the main www.cornell.edu connection. Filtering on the TLS Server Name Indication (SNI) for the page-load time window and summing the associated TCP/QUIC streams gives:

```
Total data received ≈ 997,785 bytes (≈ 975 KB / 0.97 MB)
```

Note: This capture also contained unrelated background traffic from other applications on the machine (e.g. Office 365, OneDrive, Bing) running at different timestamps; these were identified by their SNI names and excluded from the total above, since they are not part of the cornell.edu page load.

Question 3 [6 marks] — Analysis of http://neverssl.com/

A. HTTP GET requests observed

A total of **3 HTTP GET requests** were observed on the same TCP connection:

- GET /online (HTTP/1.1) — returned 301 redirect
- GET /online/ (HTTP/1.1) — returned 200 OK, main page
- GET /favicon.ico (HTTP/1.1) — returned 200 OK

B. HTTP version of the server

The server responds as **HTTP/1.1** (Server header: Apache/2.4.66).

C. TCP segments and data received per GET request

GET Request	TCP Segments Received	Data in HTTP Response
GET /online (301 Redirect)	1 segment	Content-Length: 296 bytes (575 bytes incl. headers)
GET /online/ (200 OK)	2 segments	~1519 bytes total payload (headers + HTML body)
GET /favicon.ico (200 OK)	1 segment	Content-Length: 124 bytes (416 bytes incl. headers)

D. TCP three-way handshake

Handshake observed between client port 54881 and server port 80 for the neverssl.com connection:

Step	Frame No.	Flags	Sequence No. (relative)	Ack No. (relative)
1. SYN	2331	SYN	0	—
2. SYN-ACK	2335	SYN, ACK	0	1
3. ACK	2336	ACK	1	1

(Relative sequence numbers are shown, as displayed by Wireshark by default. An earlier SYN, frame 2288, was a retransmission of the client's SYN and did not receive a reply before frame 2331 was sent.)

Tools used: Wireshark 4.2 / tshark (command-line) for packet capture inspection and statistics.